

# BotSolana

## Sistema de Copy Trading Autónomo en Solana

Sistema de inteligencia artificial para trading algorítmico que monitorea wallets profesionales en tiempo real, replica sus operaciones con gestión dinámica de riesgo y opera en modo autónomo usando patrones estadísticos derivados de 4,913 trades históricos.

Fecha del Reporte: 25 de May de 2026

Versión: 3.0 — Mayo 2026 | Deploy: Railway Production

✓ Copy Trading ✓ Modo Autónomo ✓ Groq AI Scorer ✓ Anti-pérdida

|                      |                   |                        |                     |
|----------------------|-------------------|------------------------|---------------------|
| 11                   | 4,913             | \$50 → \$2,029         | 53%                 |
| Wallets monitoreadas | Trades históricos | ROI Simulado (Railway) | Win Rate Real (SIM) |

## ■ 1. Resumen Ejecutivo

BotSolana es un sistema de trading algorítmico avanzado que opera en la blockchain Solana, combinando dos estrategias complementarias: **Copy Trading** (replica transacciones de wallets profesionales selectas) y **Trading Autónomo** (detecta tokens nuevos en Pump.fun y toma decisiones independientes basadas en patrones estadísticos). El sistema está desplegado en Railway en modo 24/7 y actualmente opera en simulación con validación de parámetros.

| Aspecto           | Detalle                                                 |
|-------------------|---------------------------------------------------------|
| Estado actual     | SIMULACIÓN activa (AUTONOMOUS_MODE=true)                |
| Capital simulado  | \$50 inicial → \$2,029 (Railway anterior, +3,958% ROI)  |
| Modo principal    | Autónomo + Copy Trading (11 wallets monitoreadas)       |
| Scorer de trading | Stat Scorer estadístico (4,913 trades) + Groq AI scorer |
| Infraestructura   | Railway (producción) + GitHub (CI/CD)                   |
| Lenguaje          | Python 3.12 con asyncio, WebSockets, Jupiter API v6     |
| Objetivo live     | Capital mínimo \$200 + win rate >65% constante en SIM   |

### ◆ Hitos alcanzados

- ■ **Copy Trading funcional:** Detecta y replica swaps de 11 wallets en <1s
- ■ **Modo Autónomo activo:** AUTONOMOUS\_MODE=true desde 21 mayo 2026
- ■ **Scorer estadístico:** 4,913 trades analizados, patrones documentados
- ■ **Groq AI scorer:** llama-3.3-70b-versatile con patrones por wallet
- ■ **3 protecciones:** Anti-pérdida de fees, price impact, liquidez mínima
- ■ **Simulador realista:** Slippage dinámico, market impact, fail rate, métricas avanzadas
- ■ **Drift logging:** Comparación SIM vs LIVE, analyze\_drift.py
- ■ **Live mode:** Pendiente: esperar WR >65% estable + capital \$200+

## ■ 2. ¿Qué es BotSolana?

BotSolana nació como un bot de **copy trading** para la blockchain Solana. La premisa es simple pero poderosa: en lugar de intentar predecir el mercado desde cero, se monitorean 11 wallets de traders profesionales con historial comprobado de 62-63% win rate. Cuando estos profesionales ejecutan un swap, el bot replica la misma operación en proporción al capital disponible.

Con el tiempo, el sistema evolucionó para incluir un **modo autónomo** que no depende de copiar wallets externas, sino que analiza cada token nuevo creado en Pump.fun y decide si comprarlo basándose en un scorer estadístico entrenado con 4,913 trades históricos. Ambos modos operan simultáneamente y comparten el mismo executor, simulador y sistema de riesgo.

### ◆ Concepto: Copy Trading

| P<br>a<br>s<br>o | Acción                                                        | Latencia                |
|------------------|---------------------------------------------------------------|-------------------------|
| 1                | Theo (wallet profesional) compra Token ABC en Pump.fun        | —                       |
| 2                | Helius WS o PumpPortal WS detecta la transacción              | 0.5 – 1 s               |
| 3                | Watcher.py filtra: ¿es una de las 11 wallets objetivo?        | < 50 ms                 |
| 4                | Executor.py calcula monto proporcional al % que invirtió Theo | < 50 ms                 |
| 5                | Scorer evalúa el token (stat_scorer + groq_scorer)            | 100 – 500 ms            |
| 6                | SIM: Simulator calcula P&L.; LIVE: TX real a blockchain       | SIM <10ms / LIVE 15-20s |
| 7                | Theo vende → bot replica la venta automáticamente             | 0.5 – 1 s               |

## ◆ Concepto: Modo Autónomo

El modo autónomo opera de forma completamente independiente. Se suscribe a PumpPortal WebSocket para recibir **todos los tokens nuevos** creados en Pump.fun en tiempo real. Cada token nuevo es trackeado durante los primeros minutos para acumular señales, y luego evaluado con el scorer estadístico. Si supera el umbral, se abre una posición que es monitoreada continuamente hasta que se activa stop-loss, take-profit, trailing stop o timeout.

| Variable             | Valor    | Descripción                                   |
|----------------------|----------|-----------------------------------------------|
| AUTO_EVAL_DELAY_MIN  | 7 min    | Espera antes de evaluar token nuevo           |
| AUTO_MOMENTUM_BUYS   | 150 buys | Trigger de evaluación anticipada por momentum |
| AUTO_STOP_LOSS_PCT   | -15%     | Venta forzada si cae más del 15%              |
| AUTO_TAKE_PROFIT_PCT | +40%     | Venta automática al alcanzar +40%             |
| AUTO_TRAILING_PEAK   | 20%      | Activar trailing stop tras +20% de ganancia   |
| AUTO_TRAILING_DROP   | 10%      | Cerrar si cae 10% desde el pico (trailing)    |
| AUTO_MAX_HOLD_MIN    | 12 min   | Timeout máximo por posición                   |
| AUTO_MAX_POSITIONS   | 3        | Posiciones autónomas simultáneas máximas      |

### 3. Arquitectura del Sistema

El sistema está organizado en capas bien definidas con responsabilidades claras. La arquitectura favorece la separación de intereses y permite que el modo copy trading y el modo autónomo compartan la capa de ejecución y simulación sin duplicar código.

| CAPA             | MÓDULO                  | RESPONSABILIDAD                                                |
|------------------|-------------------------|----------------------------------------------------------------|
| Blockchain       | Solana Mainnet          | Red donde ocurren las transacciones reales                     |
| Entrada de datos | watcher.py              | Helius WS + PumpPortal WS — detecta swaps en <1s               |
| Entrada de datos | autonomous_scanner.py   | PumpPortal WS — detecta tokens nuevos en Pump.fun              |
| Scoring          | stat_scorer.py          | Scorer determinista (4,913 trades) — sin dependencias externas |
| Scoring          | scorer.py (Groq)        | Scorer IA con patrones por wallet (llama-3.3-70b)              |
| Scoring          | learner.py              | Aprende de trades pasados en tiempo real                       |
| Ejecución        | executor.py             | Calcula monto, aplica protecciones, enruta a SIM o LIVE        |
| Simulación       | simulator.py            | P&L; realista con slippage dinámico, market impact, fees       |
| Ejecución live   | utils/pumpfun.py        | Builds TXs para Pump.fun/PumpSwap con 3-backend fallback       |
| Ejecución live   | utils/jupiter.py        | Jupiter API v6 — fallback DEX routing                          |
| Datos de mercado | utils/dexscreener.py    | Precios, liquidez, market cap en tiempo real                   |
| Datos de mercado | utils/market_context.py | Contexto de mercado para el scorer                             |
| Config           | config.py               | 60+ variables centralizadas (lee de .env/Railway)              |
| Persistencia     | data/*.json             | Posiciones abiertas, historial, balance (Railway filesystem)   |

#### ◆ Flujo de datos completo

| FLUJO DE DATOS — BOTSOLANA                                   |
|--------------------------------------------------------------|
| SOLANA BLOCKCHAIN → [HELIUS WS + PUMPPORTAL WS] → watcher.py |
| PumpPortal WS (nuevos tokens) → autonomous_scanner.py        |
| watcher.py → ¿wallet en TARGET_WALLETS? → executor.py        |
| autonomous_scanner.py → score_token() → executor.py          |
| executor.py → [Protecciones] → ¿LIVE_MODE?                   |
| Si SIM → simulator.py → P&L; teórico → logs                  |
| Si LIVE → pumpfun.py/jupiter.py → TX real → blockchain       |

## ■ 4. Módulos en Detalle

---

### ◆ `copytrade/watcher.py` — Detector de transacciones

- Conecta simultáneamente a **Helius WebSocket** (latencia 1-3s) y **PumpPortal WebSocket** (latencia 0.5s).
- Parsea todos los logs de transacciones de Solana buscando eventos de swap en programas conocidos: Jupiter v6, Raydium AMM, Orca Whirlpool, Pump.fun BC, PumpSwap AMM.
- Filtra en tiempo real: solo pasan swaps de las 11 wallets en TARGET\_WALLETS.
- Para swaps de PumpPortal WS, marca source=pumpportal para activar el fast copy path en executor (skip DexScreener+scorer → latencia <1s).
- Reutiliza reconexión exponencial con jitter para resistir caídas de WebSocket.

### ◆ `copytrade/executor.py` — Motor de ejecución

- Punto central que recibe swaps tanto de **watcher.py** (copy trade) como de **autonomous\_scanner.py** (modo autónomo).
- Aplica **6 chequeos de seguridad** en secuencia antes de ejecutar: posición duplicada, cooldown, intentos fallidos, liquidez mínima, price impact, circuit breaker.
- Calcula el monto a invertir de forma proporcional: si Theo invirtió 10% de su capital, el bot invierte 10% del suyo. Respeta TRADE\_CAP\_TIERS y SCALING\_TIERS.
- Modo SIM: delega a simulator.py. Modo LIVE: construye TX real via pumpfun.py o jupiter.py con 3-backend fallback.
- Registra trades en data/copytrades.json para análisis posterior.

### ◆ `copytrade/simulator.py` — Simulador realista

- Replica con alta fidelidad lo que ocurriría en live mode, aplicando **5 capas de realismo cuantitativo**.
- **Realismo 1 — Latencia**: aplica penalidad de  $1.5s \times 1.5\%/s = +2.25\%$  en precio de entrada.
- **Realismo 2 — Slippage dinámico**: varía según ratio trade\_usd/liquidity\_usd, cap 30% en compra, 50% en venta.
- **Realismo 3 — Market impact no lineal**:  $\text{impacto} = \sqrt{(\text{trade}/\text{liq})} \times 0.35$ , cap 40%.
- **Realismo 4 — TX fail rate inteligente**: 8% base + penalidad por liquidez baja + volatilidad.
- **Realismo 5 — Métricas avanzadas**: profit factor, expectancy, max drawdown, Sharpe ratio.
- Escala automáticamente el tamaño de trade con TRADE\_CAP\_TIERS según balance.
- Registra cada trade en execution\_drift.jsonl para análisis de divergencia SIM vs LIVE.

### ◆ `copytrade/autonomous_scanner.py` — Scanner autónomo

- Se suscribe a PumpPortal WS con subscribeNewToken para recibir cada token nuevo creado en Pump.fun en tiempo real.
- Por cada token nuevo, se suscribe también a subscribeTokenTrade para acumular buys/sells y datos de bonding curve durante los primeros minutos.

- Evaluación programada a los 7 minutos (AUTO\_EVAL\_DELAY\_MIN), pero con **momentum trigger anticipado**: si acumula  $\geq 150$  buys, evalúa inmediatamente.
- Fetch en cascada para obtener datos del token: DexScreener → PumpPortal API → datos WS acumulados (garantiza evaluación aunque fallen las APIs externas).
- Monitor de precio asíncrono por posición (asyncio.Task): chequea cada 10s aplicando SL/TP/trailing/timeout.

# ■ 5. Sistema de Scoring — Inteligencia del Bot

El corazón del sistema autónomo es el **scorer**, que decide qué tokens merecen una posición y cuáles se descartan. BotSolana implementa dos scorers complementarios:

## ◆ 5.1 Stat Scorer — Determinista

Basado en el análisis estadístico de **4,913 trades históricos** reales de las wallets monitoreadas. No requiere API externa — siempre disponible, respuesta instantánea. Evalúa 6 dimensiones con pesos derivados de win rates medidos:

| Dimensión      | Rango óptimo    | Score   | Win Rate base |
|----------------|-----------------|---------|---------------|
| Edad del token | 5 – 15 minutos  | +25 pts | 60.1%         |
| Edad del token | 15 – 30 minutos | +15 pts | 51.5%         |
| Edad del token | 60+ minutos     | −30 pts | 27.6% ■       |
| Liquidez (USD) | \$2k – \$10k    | +20 pts | 48.5%         |
| Liquidez (USD) | < \$500         | −15 pts | bajo ■        |
| Market Cap     | \$5k – \$20k    | +20 pts | 53.5%         |
| Market Cap     | > \$100k        | −10 pts | 39.0% ■       |
| Cambio 1h      | > +200%         | +25 pts | 70.5% ■       |
| Cambio 1h      | −50% a 0%       | +10 pts | 48.2%         |
| Cambio 1h      | +50% a +200%    | −20 pts | 16.7% ■       |
| Buys 5 min     | ≥ 200 buys      | +25 pts | 68.1% ■       |
| Buys 5 min     | 50 – 200 buys   | +15 pts | 59.4%         |
| Buys 5 min     | 10 – 50 buys    | −5 pts  | 36.0% ■       |
| Programa DEX   | Raydium         | +20 pts | 79.2% ■       |
| Programa DEX   | PumpSwap        | +5 pts  | 42.5%         |
| Programa DEX   | Jupiter         | −50 pts | 0.0% ■        |

**Umbral de compra:** score ≥ 50 puntos (configurable con SCORER\_THRESHOLD). Score máximo teórico: ~115 puntos. En SIM, el scorer opera en modo observación (no bloquea trades) para continuar recolectando datos sin sesgo.

## ◆ 5.2 Groq AI Scorer — Patrones por wallet

Utiliza **Groq llama-3.3-70b-versatile** para analizar patrones específicos por wallet. El pipeline de aprendizaje consiste en 4 etapas:

| Etap         | Script                          | Descripción                                                                        |
|--------------|---------------------------------|------------------------------------------------------------------------------------|
| 1 — Descarga | data_collector/fetch_history.py | Descarga historial on-chain (11 wallets, 14 días, cap 1,000 firmas) via Helius RPC |

| Etapa          | Script                             | Descripción                                                                 |
|----------------|------------------------------------|-----------------------------------------------------------------------------|
| 2 — Outcomes   | data_collector/compute_outcomes.py | Detecta cuándo se vendió cada token y calcula WIN/LOSS real                 |
| 3 — Análisis   | data_collector/groq_analyzer.py    | Groq analiza patrones de WIN vs LOSS por wallet (edad, liquidez, DEX, etc.) |
| 4 — Aplicación | copytrade/scorer.py                | Evalúa tokens en tiempo real con score 0-100, threshold 40                  |

### ■ Resultados del análisis Groq (2,487 trades, 8 wallets)

| Wallet   | Win Rate | Patrón detectado                                                         |
|----------|----------|--------------------------------------------------------------------------|
| Cupsey-2 | 63%      | < 10 min edad, liq \$1k-\$10k, Pump.fun                                  |
| Theo     | 63%      | < 10 min edad, hold largo (da tiempo de entrada)                         |
| Cented   | 62%      | < 10 min, liq \$1k-\$10k, Pump.fun                                       |
| Trey     | 62%      | < 10 min, patrones similares (■ hold corto, solo 1 trade medido)         |
| Domy     | 61%      | < 10 min, hold < 1 min, Pump.fun                                         |
| Cupsey   | 58%      | 1-10 min, liq \$1k+, PumpSwap                                            |
| Decu     | 56%      | < 10 min, liq \$1k-\$10k                                                 |
| Nyhrox   | 55%      | 0-1 min hold ■ — trampa parcial, moves terminan antes de nuestra entrada |



## ■ 6. Gestión de Riesgo y Protecciones

El sistema implementa múltiples capas de protección para minimizar pérdidas, especialmente críticas en el mercado de microcaps de Pump.fun donde la volatilidad y la falla de transacciones son comunes.

### ◆ 6.1 Protecciones en executor.py

| # | Protección         | Condición                       | Acción                                  |
|---|--------------------|---------------------------------|-----------------------------------------|
| 1 | Posición duplicada | Token ya tiene posición abierta | Ignorar (no abrir 2x)                   |
| 2 | Cooldown 2 min     | Token vendido hace < 2 min      | Ignorar (evita trades reentrada rápida) |
| 3 | Failed attempts    | Token falló $\geq 2$ veces      | Ignorar (ahorra fees)                   |
| 4 | Liquidez mínima    | Liquidez DexScreener < \$500    | Abortar (slippage extremo)              |
| 5 | Price impact       | Price impact > 50% (Jupiter)    | Abortar (TX fallaría on-chain)          |
| 6 | Circuit breaker    | Pérdida sesión > 20%            | Detener TODOS los trades                |

### ◆ 6.2 Protecciones en modo autónomo

| Mecanismo      | Trigger                                    | Descripción                                          |
|----------------|--------------------------------------------|------------------------------------------------------|
| Stop Loss      | -15%                                       | Venta inmediata si precio cae 15% desde entrada      |
| Take Profit    | +40%                                       | Cierre de posición al alcanzar ganancia del 40%      |
| Trailing Stop  | Pico $\geq +20\%$ $\rightarrow$ caída -10% | Protege ganancias: si sube +30% y cae a +20%, cierra |
| Timeout        | 12 minutos máx                             | Cierre forzado si la posición no se resuelve         |
| Max posiciones | 3 simultáneas                              | Evita sobreexposición con muchas posiciones abiertas |

### ◆ 6.3 Gestión dinámica de capital

El tamaño de cada trade escala automáticamente con el balance disponible, asegurando que el riesgo absoluto crece proporcionalmente a las ganancias:

| Balance       | % por trade | Tope USD por trade | Razonamiento                       |
|---------------|-------------|--------------------|------------------------------------|
| \$0 – \$100   | 10%         | \$5 máx            | Capital bajo: minimizar exposición |
| \$100 – \$300 | 10%         | \$15 máx           | Fase de construcción               |
| \$300 – \$600 | 10%         | \$35 máx           | Balance estable                    |
| \$600 – \$1k  | 10%         | \$60 máx           | ← Nivel Railway anterior           |
| \$1k – \$2k   | 7%          | \$90 máx           | Reducción % al crecer              |
| \$2k – \$5k   | 7%          | \$130 máx          | Diversificación implícita          |
| \$5k+         | 3%          | \$200 máx          | Capital grande: % mínimo           |

## ◆ 6.4 Stop Loss global y circuit breaker

Además de las protecciones por trade, existe un **stop loss global** (STOP\_LOSS\_PCT=0.70): si el balance cae por debajo del 70% del capital inicial, el bot deja de operar. El **circuit breaker de sesión** (MAX\_SESSION\_LOSS\_PCT=0.20) para todos los trades si se pierde más del 20% del balance en la sesión actual, requiriendo reinicio manual para reactivar.

## ■ 7. Stack Tecnológico

### ◆ 7.1 Lenguaje y runtime

| Tecnología    | Versión     | Uso                                                               |
|---------------|-------------|-------------------------------------------------------------------|
| Python        | 3.12 / 3.10 | Lenguaje principal del bot                                        |
| asyncio       | Estándar    | Concurrencia — watcher + scanner + monitor de precios en paralelo |
| websockets    | ≥12.0       | Conexión a Helius WS y PumpPortal WS                              |
| httpx         | ≥0.27.0     | Requests HTTP async a DexScreener, Jupiter, PumpPortal            |
| solana-py     | ≥0.34.0     | Cliente RPC para Solana blockchain                                |
| solders       | ≥0.21.0     | Construir y firmar transacciones Solana                           |
| rich          | Última      | TUI en terminal — logs coloridos, tablas, paneles                 |
| python-dotenv | Última      | Carga variables de entorno desde .env                             |
| certifi       | Última      | Certificados SSL para WebSocket seguro                            |

### ◆ 7.2 APIs y servicios externos

| Servicio        | Propósito                                                         | Tipo               |
|-----------------|-------------------------------------------------------------------|--------------------|
| Helius RPC      | Nodo HTTP + WebSocket de Solana. Alta disponibilidad              | RPC pago           |
| PumpPortal WS   | Detección de swaps en Pump.fun con 0.5s de latencia               | WebSocket gratuito |
| PumpPortal API  | Precio de tokens en bonding curve cuando DexScreener no los tiene | REST gratuito      |
| DexScreener API | Precios, liquidez, market cap, cambios de precio                  | REST gratuito      |
| Jupiter API v6  | Quotes de swap y ejecución. Fallback robusto para todos los DEX   | REST gratuito      |
| Groq API        | Análisis de patrones con llama-3.3-70b-versatile (scorer IA)      | REST pago          |
| CoinGecko       | Precio SOL en USD (cache 60s)                                     | REST gratuito      |
| Railway         | Hosting 24/7, variables de entorno, logs                          | PaaS pago          |
| GitHub          | Control de versiones + CI/CD (deploy automático en push)          | Git gratuito       |

### ◆ 7.3 Infraestructura de deploy

**Railway** es el servidor de producción. El bot corre como un proceso Python continuo (definido en Procfile: worker: python3 main.py). Las credenciales sensibles (WALLET\_PRIVKEY\_B58, GROQ\_API\_KEY, etc.) se almacenan como variables de entorno en Railway, nunca en el código ni en el repositorio.

**Limitación conocida:** El filesystem de Railway es efímero — el directorio data/ (balance, historial, posiciones abiertas) se borra en cada redeploy. Para persistencia real se necesita un Railway Volume o base de datos externa. Por ahora se acepta el reset, y el bot arranca desde SIM\_CAPITAL en cada deploy.

## ■ 8. Wallets Monitoreadas

El sistema monitorea 11 wallets de traders profesionales con historial comprobado. Cada wallet tiene un label para identificarla en los logs. La **asignación de capital es ponderada** según win rate histórico, con Cupsey-2 recibiendo el mayor peso (40%) por su consistencia.

| Label    | Dirección (primeros 20 chars...) | Win Rate | Peso capital | Notas                                     |
|----------|----------------------------------|----------|--------------|-------------------------------------------|
| Cented   | CyaE1VxvBrahnPWkqm5V...          | 62%      | 20%          | Incluida en análisis Groq                 |
| Domy     | 3LUfv2u5yzsDtUzPdsSJ...          | 61%      | —            | Hold < 1 min, Pump.fun                    |
| Theo     | Bi4rd5FH5bYEN8scZ7we...          | 63%      | —            | Mejor para copy: hold largo               |
| Cupsey ■ | 2fg5QD1eD7rzNNCsvn...            | 58%      | 10%          | 1-10 min, PumpSwap                        |
| Nyhrox   | 6S8GezkxYUfZy9JPtYna...          | 55%      | —            | ■ Trampa: moves terminan antes de entrada |
| Cupsey-2 | 4BdKaxN8G6ka4GYtQQWk...          | 63%      | 40%          | ★ Mejor rendimiento — mayor peso          |
| Decu     | 4vw54BmAogeRV3vPKWyF...          | 56%      | 30%          | Estable, buen historial                   |
| Orange   | DuQabFqdC9eeBULVa7TT...          | —        | —            | En monitoreo                              |
| Insentos | 7SDs3PJt2mswKQ7Z04FT...          | —        | —            | En monitoreo                              |
| Trey     | 831y hv67QpKqLBJbmw2...          | 62%      | —            | ■ No copiar hasta 50+ trades de historial |
| RC       | DxM1hfY8FQ8dNGrucuJz...          | —        | —            | En monitoreo                              |

**Nota sobre Nyhrox:** Análisis del 11 mayo 2026 reveló que Nyhrox opera con holds de 0-1 minuto, lo que significa que la venta ocurre antes de que el bot pueda ejecutar la compra y aprovechar el movimiento. Se mantiene en monitoreo pero se recomienda no asignarle capital hasta confirmar mejoras de latencia.

## ■ 9. Historial de Desarrollo

El proyecto evolucionó iterativamente, con cada sesión de desarrollo respondiendo a problemas concretos identificados en logs y análisis de resultados.

| Fecha       | Versión/Commit | Cambio                                                                                                    |
|-------------|----------------|-----------------------------------------------------------------------------------------------------------|
| Marzo 2026  | Initial        | Bot de copy trading básico: Helius WS + Jupiter + 3 wallets                                               |
| 26 Abr 2026 | múltiples      | PumpPortal WS (latencia 0.5s), fee reducida 0.0005→0.0002, MAX_OPEN=5, slippage 20→15%                    |
| 29 Abr 2026 | múltiples      | Expansión a 11 wallets monitoreadas, simulador realista (fees + slippage deducidos)                       |
| 3 May 2026  | múltiples      | TRADE_CAP_TIERS implementado, SIM_CAPITAL=\$667, SIM_RESET=false                                          |
| 6 May 2026  | 61a092d        | ■ Live attempt fallido (PumpPortal HTTP 400). Implementadas 3 protecciones anti-pérdida                   |
| 6 May 2026  | d769fe7        | Circuit breaker de sesión (MAX_SESSION_LOSS_PCT=0.20)                                                     |
| 6 May 2026  | a1bceaa        | Cooldown de 2 min anti-reentrada rápida                                                                   |
| 7 May 2026  | 4d254c3        | Realismo brutal: slippage dinámico, market impact $\sqrt{\phantom{x}}$ , TX fail rate, métricas Sharpe/PF |
| 10 May 2026 | múltiples      | live_micro.py: live trading con capital micro (\$50), análisis Railway                                    |
| 11 May 2026 | 2a7dfceb       | Execution drift logging (SIM vs LIVE), analyze_drift.py                                                   |
| 12 May 2026 | ca0b696        | Groq AI scorer completo: fetch_history + compute_outcomes + groq_analyzer + scorer.py                     |
| 21 May 2026 | 30bad15        | Fix precio autónomo: _fetch_pumpportal_price(), last_price_usd, fast copy path                            |
| 21 May 2026 | b065b93        | AUTONOMOUS_MODE=true activado, stat_scorer.py (4,913 trades, sin Groq)                                    |

# 10. Resultados y Métricas

## 10.1 Mejores resultados en simulación (Railway)

El resultado más significativo en Railway (antes de reinicio por redeploy) fue:

| Métrica              | Valor                                           | Contexto                                       |
|----------------------|-------------------------------------------------|------------------------------------------------|
| Capital inicial      | \$50.00                                         | SIM_CAPITAL en Railway                         |
| Balance máximo       | \$2,029.00                                      | Antes de reinicio por redeploy                 |
| ROI simulado         | +3,958%                                         | Con filesystem efímero (sin persistencia real) |
| Total trades         | 215 trades                                      | En sesión continua                             |
| Win rate             | 53%                                             | 114 wins / 101 losses                          |
| Wallets activas      | 4                                               | Theo, Nyhrox, Cupsey-2, Trey                   |
| Mega-wins destacados | Theo: +104.6%, Nyhrox: +162.4%, Nyhrox: +119.6% | 5VBRhr, 2P6ZGj, FUKQZq                         |

## 10.2 Análisis de win rates históricos por wallet

El análisis Groq procesó 2,487 trades de 8 wallets en 14 días. Los resultados confirman que las mejores wallets son consistentemente superiores al 60% WR:

| Wallet   | Trades | Win Rate | Mejor patrón                      | Recomendación                    |
|----------|--------|----------|-----------------------------------|----------------------------------|
| Cupsey-2 | ~300+  | 63%      | <10 min, liq \$1k-\$10k, Pump.fun | ■ 40% del capital                |
| Theo     | ~300+  | 63%      | <10 min, hold largo               | ■ Prioritaria                    |
| Cented   | ~300+  | 62%      | <10 min, Pump.fun                 | ■ 20% del capital                |
| Trey     | ~300+  | 62%      | <10 min                           | ■ Solo 1 trade medido en Railway |
| Domy     | ~300+  | 61%      | <10 min, hold <1 min              | ■ Monitorear                     |
| Cupsey   | ~300+  | 58%      | 1-10 min, PumpSwap                | ■ 10% del capital                |
| Decu     | ~300+  | 56%      | <10 min, liq \$1k-\$10k           | ■ 30% del capital                |
| Nyhrox   | ~300+  | 55%      | 0-1 min hold                      | ■ Trampa de latencia             |

## 10.3 Métricas avanzadas del simulador

El simulador calcula métricas profesionales de trading cuantitativo para evaluar la calidad real de la estrategia, más allá del simple win rate:

| Métrica       | Fórmula                        | Objetivo | Interpretación                                                           |
|---------------|--------------------------------|----------|--------------------------------------------------------------------------|
| Profit Factor | Suma ganancias / Suma pérdidas | > 1.3    | Cuánto gana por cada \$ que pierde. PF=2 = gana \$2 por cada \$1 perdido |

| Métrica        | Fórmula                                | Objetivo | Interpretación                                 |
|----------------|----------------------------------------|----------|------------------------------------------------|
| Expectancy     | $WR \times AvgWin - LR \times AvgLoss$ | > \$0    | Ganancia esperada por trade. El verdadero edge |
| Max Drawdown   | Máxima caída pico → valle              | < 25%    | Peor momento de la equity curve                |
| Sharpe Ratio   | $E[PnL\%] / \sigma(PnL\%)$             | > 1.0    | Retorno ajustado por riesgo                    |
| Win/Loss Ratio | $AvgWin / AvgLoss$                     | > 1.5    | Cuánto gana en promedio vs cuánto pierde       |

## ■ 11. Variables de Configuración (Railway)

Todas las variables se configuran como **Environment Variables en Railway**. El bot nunca almacena credenciales en el código. config.py las lee en tiempo de inicio.

### ◆ Wallet y Blockchain

| Variable           | Valor actual                                | Descripción                               |
|--------------------|---------------------------------------------|-------------------------------------------|
| WALLET_PUBKEY      | F9kYAERneG7Q...                             | Dirección pública de la wallet de trading |
| WALLET_PRIVKEY_B58 | ***SECRETO***                               | Clave privada en base58 — NUNCA compartir |
| SOLANA_RPC_HTTP    | https://mainnet.helius-rpc.com/?api-key=... | Nodo HTTP Solana (Helius)                 |
| SOLANA_RPC_WS      | wss://mainnet.helius-rpc.com/?api-key=...   | Nodo WebSocket Solana (Helius)            |

### ◆ Modo y Capital

| Variable        | Valor actual          | Descripción                                |
|-----------------|-----------------------|--------------------------------------------|
| LIVE_MODE       | false                 | false=simulación   true=trading real       |
| AUTONOMOUS_MODE | true                  | Activar modo autónomo (sin TARGET_WALLETS) |
| SIM_CAPITAL     | 50                    | Capital inicial en simulación (USD)        |
| SIM_RESET       | false                 | true=borrar datos al reiniciar             |
| TARGET_WALLETS  | CyaE1Vx...,3LUfv2u... | Wallets a copiar (separadas por coma)      |

### ◆ Scorer

| Variable              | Valor actual | Descripción                                  |
|-----------------------|--------------|----------------------------------------------|
| USE_GROQ_SCORER       | true         | Activar scorer IA (reemplaza ONLY_AMM_SWAPS) |
| SCORER_THRESHOLD      | 40           | Score mínimo para copiar (0-100)             |
| GROQ_API_KEY          | gsk_***      | API key de Groq para scorer IA               |
| SCORER_ENFORCE_IN_SIM | false        | false=observa sin bloquear en SIM            |

### ◆ Riesgo y Protecciones

| Variable             | Valor actual | Descripción                                    |
|----------------------|--------------|------------------------------------------------|
| MAX_TRADE_PCT        | 0.035        | % máximo del balance por trade (3.5%)          |
| STOP_LOSS_PCT        | 0.70         | Parar si balance < 70% del capital inicial     |
| MAX_SESSION_LOSS_PCT | 0.20         | Circuit breaker: parar si pérdida sesión > 20% |



| Variable          | Valor actual | Descripción                            |
|-------------------|--------------|----------------------------------------|
| MAX_PRICE_IMPACT  | 2.0          | Price impact máximo permitido (2%)     |
| MIN_LIQUIDITY_USD | 500          | Liquidez mínima en DexScreener (\$500) |

### ◆ Simulador

| Variable             | Valor actual | Descripción                                 |
|----------------------|--------------|---------------------------------------------|
| SIM_SLIPPAGE_PCT     | 0.015        | Slippage por operación (1.5%)               |
| SIM_PRIORITY_FEE_SOL | 0.0004       | Fee round-trip en SOL ( $2 \times 0.0002$ ) |
| SIM_BASE_FAIL_RATE   | 0.08         | Tasa de fallo base de TXs (8%)              |
| SIM_MAX_HOLD_MIN     | 10000        | Auto-close posiciones sin señal de venta    |

### ◆ Modo Autónomo

| Variable             | Valor actual | Descripción                          |
|----------------------|--------------|--------------------------------------|
| AUTO_EVAL_DELAY_MIN  | 7            | Minutos antes de evaluar token nuevo |
| AUTO_MOMENTUM_BUYS   | 150          | Buys para trigger anticipado         |
| AUTO_STOP_LOSS_PCT   | -15          | Stop loss autónomo (−15%)            |
| AUTO_TAKE_PROFIT_PCT | 40           | Take profit autónomo (+40%)          |
| AUTO_MAX_POSITIONS   | 3            | Posiciones autónomas simultáneas     |

## ■ 12. Incidentes y Lecciones Aprendidas

### ■ Live Attempt Fallido — 6 Mayo 2026

- Capital: \$60 reales (0.1556 SOL)
- Causa: PumpPortal API respondía HTTP 400 en todos los requests durante el deploy
- Resultado: 0 trades ejecutados. Capital preservado (\$56 al retirar — pérdida solo por caída precio SOL)
- Lección 1: Siempre verificar API externa con 5+ tests exitosos antes de activar live
- Lección 2: Capital mínimo para live mode: \$200 (no \$60) — con \$60, las fees son >5% del trade
- Lección 3: Con capital pequeño, 1 trade fallido = pérdida significativa de %

### ■ Filesystem Efímero Railway — Mayo 2026

- Problema: data/ se borra en cada redeploy → balance, historial y posiciones se pierden
- Impacto: ROI +3,958% se resetea a \$50 en cada deploy
- Solución temporal: SIM\_RESET=false evita borrado en reinicios normales
- Solución definitiva pendiente: Railway Volume persistente o DB externa (SQLite/PostgreSQL)

### ■ Precio = 0 en Modo Autónomo — Mayo 2026

- Problema: Tokens de Pump.fun BC sin indexar en DexScreener → precio retornaba 0
- Consecuencia: P&L; siempre 0%, SL/TP nunca se activaban, todo cerraba por timeout
- Solución: `_fetch_pumpportal_price()` como fallback, `last_price_usd` actualizado en cada tick
- Commit: 30bad15 (21 mayo 2026)

### ■ Latencia Nyhrox — Análisis 11 Mayo 2026

- Problema: Nyhrox opera con holds de 0-1 minuto → bot no puede entrar y salir antes de que termine el move
- Evidencia: trade 215 con Trey: 4dd4Uw -50.5% (-\$98.48) inmediato al agregar wallet nueva
- Lección: No agregar wallets sin ≥50 trades de historial verificado en SIM

## ■ 13. Roadmap y Estado Actual

### ◆ 13.1 Condiciones para activar Live Mode

Las siguientes condiciones deben cumplirse **todas** antes de activar live trading real:

| # | Condición                                                                   | Estado         |
|---|-----------------------------------------------------------------------------|----------------|
| 1 | Win rate > 65% consistente en SIM durante 1-2 semanas                       | ■ En progreso  |
| 2 | Capital mínimo \$200 (fees < 2% del trade con \$200 y 0.0004 SOL fee)       | ■ Pendiente    |
| 3 | PumpPortal API: 5+ test trades exitosos verificados manualmente             | ■ Pendiente    |
| 4 | Jupiter API disponible y respondiendo correctamente                         | ■ Pendiente    |
| 5 | 2FA activado en Railway y GitHub                                            | ■ Pendiente    |
| 6 | Wallet de trading separada del resto de fondos (wallet fría para ganancias) | ■ Pendiente    |
| 7 | Circuit breaker configurado y probado en SIM                                | ■ Implementado |
| 8 | Railway Volume para persistencia (o aceptar reset consciente)               | ■ Pendiente    |

### ◆ 13.2 Próximas mejoras planificadas

| Prioridad | Mejora                                                 | Impacto                                    |
|-----------|--------------------------------------------------------|--------------------------------------------|
| Alta      | Railway Volume para persistir data/ entre redeploy     | Elimina reset de balance en cada deploy    |
| Alta      | Evaluación de 7 días de SIM con AUTONOMOUS_MODE=true   | Validar scorer en condiciones reales       |
| Alta      | Análisis de execution drift SIM vs LIVE                | Cuantificar divergencia real antes de live |
| Media     | Webhook de Telegram para alertas de trades             | Monitoreo en tiempo real desde móvil       |
| Media     | Dashboard web para ver equity curve en tiempo real     | Visualización de rendimiento               |
| Media     | Ampliar dataset a 30 días para reentrenar scorer       | Más precisión estadística                  |
| Baja      | Soporte Ethereum completo (eth_watcher + eth_executor) | Diversificación de red                     |
| Baja      | Weighted allocation dinámica (reweight cada 24h)       | Optimización automática de capital         |

## ■ 14. Estructura del Repositorio

| Archivo/Directorio              | Descripción                                                           |
|---------------------------------|-----------------------------------------------------------------------|
| main.py                         | Punto de entrada. Banner, UI Rich, inicia watch_all() y modo autónomo |
| config.py                       | 60+ variables centralizadas. Lee de .env / Railway env vars           |
| requirements.txt                | Dependencias Python: solana, httpx, websockets, rich, etc.            |
| Procfile                        | worker: python3 main.py (instrucción de deploy para Railway)          |
| railway.json                    | Config Railway: healthcheckPath, restartPolicyType                    |
| run.sh                          | Script local de ejecución rápida                                      |
| analyze_drift.py                | Analiza execution_drift.jsonl local — compara SIM vs LIVE             |
| compare_live_vs_sim.py          | Comparación profunda de rendimiento SIM vs LIVE                       |
| live_micro.py                   | Bot de live trading micro (capital pequeño, experimental)             |
| demo_live_micro.py              | Demo/test del live micro sin capital real                             |
| copytrade/watcher.py            | Detecta swaps: Helius WS + PumpPortal WS en paralelo                  |
| copytrade/executor.py           | Motor de ejecución con 6 protecciones + routing SIM/LIVE              |
| copytrade/simulator.py          | Simulador P&L; realista con 5 capas de realismo cuantitativo          |
| copytrade/autonomous_scanner.py | Scanner autónomo: detecta tokens nuevos en Pump.fun                   |
| copytrade/stat_scorer.py        | Scorer estadístico determinista (4,913 trades)                        |
| copytrade/scorer.py             | Scorer IA con Groq (patrones por wallet)                              |
| copytrade/learner.py            | Aprendizaje en tiempo real de trades pasados                          |
| copytrade/decoder.py            | Decodifica instrucciones de swaps en logs de Solana                   |
| copytrade/eth_watcher.py        | Watcher para red Ethereum (experimental)                              |
| copytrade/eth_executor.py       | Executor para Ethereum (experimental)                                 |
| copytrade/eth_simulator.py      | Simulador para Ethereum (experimental)                                |
| utils/dexscreener.py            | Cliente DexScreener API — precios, liquidez, market cap               |
| utils/pumpfun.py                | Builds TXs Pump.fun/PumpSwap con 3-backend fallback                   |
| utils/jupiter.py                | Jupiter API v6 — quotes + transacciones DEX                           |
| utils/market_context.py         | Contexto de mercado para el scorer                                    |
| utils/blockchain.py             | Cliente Solana RPC — balances, transacciones                          |
| utils/wallet_scoring.py         | Scoring de wallets basado en historial                                |
| utils/logger.py                 | Logger con Rich — formatos coloridos y paneles                        |
| utils/alchemy_client.py         | Cliente Alchemy para datos ETH                                        |
| utils/exit_degradation.py       | Degradación graceful al cerrar bot                                    |
| data_collector/fetch_history.py | Descarga historial on-chain (11 wallets, 14 días)                     |

| Archivo/Directorio                 | Descripción                                                             |
|------------------------------------|-------------------------------------------------------------------------|
| data_collector/compute_outcomes.py | Calcula WIN/LOSS real de cada trade histórico                           |
| data_collector/groq_analyzer.py    | Analiza patrones con Groq llama-3.3-70b                                 |
| data/                              | Runtime: posiciones, historial, balance, drift log (efímero en Railway) |
| logs/                              | Logs diarios: simulator_YYYYMMDD.log, executor_YYYYMMDD.log, etc.       |
| .env                               | Variables locales (NO subir a git — incluido en .gitignore)             |
| .env.example                       | Template de variables requeridas (sin valores)                          |

## ■ 15. Conclusiones

---

BotSolana representa un sistema de trading algorítmico maduro con arquitectura sólida, múltiples capas de protección y dos modos operativos complementarios. El sistema ha demostrado capacidad para generar resultados positivos en simulación (+3,958% ROI en el mejor run), y las mejoras de realismo en el simulador garantizan que estos resultados reflejan con alta fidelidad el comportamiento esperado en live.

Los principales puntos fuertes del sistema son:

- ✓ Arquitectura asíncrona robusta — múltiples WebSockets en paralelo con reconexión automática
- ✓ Simulador cuantitativo realista — 5 capas de realismo incluyendo market impact no lineal
- ✓ Scorer estadístico determinista — sin dependencias externas, basado en 4,913 trades reales
- ✓ Gestión dinámica de riesgo — 6 protecciones + circuit breaker + escalado progresivo de capital
- ✓ Modo autónomo funcional — opera sin wallets objetivo usando solo análisis estadístico
- ✓ Drift logging — herramienta para cuantificar divergencia SIM vs LIVE antes de arriesgar capital
- ✓ Modularidad — copiar wallet, ejecutar en SIM/LIVE y calcular P&L; son capas independientes

Los puntos a mejorar antes de escalar a live mode:

- Persistencia de datos: Railway filesystem efímero borra historial en cada redeploy
- Capital insuficiente para live: se necesita \$200+ para que las fees no sean >5% del trade
- Win rate SIM: se necesita validar >65% durante 1-2 semanas completas
- Latencia en tokens BC: tokens muy nuevos en Pump.fun tardan en indexarse en DexScreener

**Estado final:** El sistema está listo para validación extendida en simulación con `AUTONOMOUS_MODE=true`. Una vez que los resultados muestren win rate consistente > 65% durante 7-14 días, y se cuente con capital mínimo de \$200, el live mode puede activarse con alta confianza en que el simulador predijo correctamente el comportamiento.